AP CSA Class Template

Write-a-class-from-scratch skeleton, 4 variations, and the traps

The complete skeleton for "write a class from scratch" questions, plus the variations the exam actually asks for. Copy the shape, not the bank account.

1. The Full Template (copy this shape)

```
public class BankAccount {

    // 1. Instance variables: ALWAYS private
    private String owner;
    private double balance;
    private int transactions;

    // 2. Static (class) variable: ONE copy shared by all objects
    private static int totalAccounts = 0;

    // 3. Constructor: same name as the class, no return type
    public BankAccount(String owner, double balance) {
        this.owner = owner;
        this.balance = balance;
        this.transactions = 0;
        totalAccounts++;
    }

    // 4. Accessors: return a copy of the field, take no parameters
    public String getOwner() { return owner; }
    public double getBalance() { return balance; }

    // 5. Mutators / behavior: change state, return nothing (or a boolean)
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            transactions++;
        }
    }

    public boolean withdraw(double amount) {
        if (amount <= 0 || amount > balance) return false;
        balance -= amount;
        transactions++;
        return true;
    }

    // 6. Static method: NO this, NO instance fields
    public static int getTotalAccounts() { return totalAccounts; }

    // 7. toString: lets System.out.println(obj) print something readable
    public String toString() {
        return owner + ": $" + balance;
    }

    // 8. main: only when the question asks for it
    public static void main(String[] args) {
        BankAccount a = new BankAccount("Ada", 100.0);
        a.deposit(25.0);
        System.out.println(a);   // Ada: $125.0
    }
}
```

2. Anatomy: What Each Part Must Look Like

Part	Required form	Exam expectation
Class header	`public class Name`	Capitalized, matches the file name, no `()`
Instance field	`private type name;`	`private` unless told otherwise; initialize in constructor
Static field	`private static type name = 0;`	One shared copy; counts objects or holds constants
Constructor	`public Name(params)`	**No return type**, not even `void`
Getter	`public type getField()`	Returns the value; no `System.out.print` inside
Setter	`public void setField(type v)`	Assigns with `this.field = v;`
Behavior method	`public type verbNoun(params)`	Does the work described in the prompt
`toString`	`public String toString()`	Returns `String`; no parameters; no printing
`main`	`public static void main(String[] args)`	Only if asked

3. Naming and Visibility Rules (free points, easy to lose)

- **Class** -> `PascalCase`: `BankAccount`, `Song`, `StudentRecord`.
- **Fields, methods, variables** -> `camelCase`: `accountBalance`, `getBalance()`, `numItems`.
- **Fields are `private`.** AP CSA graders expect encapsulation; a `public` field in a "write a class" answer usually loses the encapsulation point.
- **Methods the question calls must be `public`.**
- **Constants**: `private static final int MAX_SIZE = 10;` - all caps, `static final`, never reassigned.

4. Six-Step Recipe for "Write a Class" Questions

1. **Underline the nouns** in the prompt -> those become fields.
2. **Underline the verbs** -> those become methods (with parameters = the things being acted on).
3. **Write the field declarations first**, all `private`, all with a type that matches the data.
4. **Write the constructor** so every field is assigned exactly once - parameters first, then `this.field = value;`.
5. **Write each method minimal**: guard clause for bad input, one loop if a collection is involved, one `return` of the right type.
6. **Add `toString`** if the prompt says "should print/represent nicely", and **test mentally with one sample object**.

> If the prompt gives a constructor signature, copy it verbatim - parameter types and order are graded literally.

5. Variation A: Subclass With `super()`

```
public class SavingsAccount extends BankAccount {
    private double interestRate;

    public SavingsAccount(String owner, double balance, double interestRate) {
        super(owner, balance);        // MUST be the first statement
        this.interestRate = interestRate;
    }

    public void addInterest() {
        deposit(getBalance() * interestRate);   // private parent fields -> use getters
    }

    @Override
    public String toString() {
        return "Savings " + super.toString() + " @" + interestRate;
    }
}
```

- `super(...)` must be line 1 of the subclass constructor.

- `private` superclass fields are **invisible** here - go through getters/mutators.
- Overriding needs the identical signature; `@Override` is optional but signals intent.

6. Variation B: Class Holding an ArrayList

```
import java.util.ArrayList;

public class Playlist {
    private String name;
    private ArrayList<String> songs;

    public Playlist(String name) {
        this.name = name;
        this.songs = new ArrayList<String>();   // MUST construct it here
    }

    public void addSong(String song) {
        songs.add(song);                        // no .length on ArrayList!
    }

    public int countLongTitles() {
        int count = 0;
        for (String s : songs) {
            if (s.length() > 20) count++;
        }
        return count;
    }

    public ArrayList<String> getSongs() {
        return new ArrayList<String>(songs);    // defensive copy: caller cannot wreck the list
    }
}
```

- Declaring `private ArrayList<String> songs;` **does not** create the list - `new` it in the constructor, or you get `NullPointerException` on the first `add`.
- `songs.size()`, not `songs.length`; `songs.get(i)`, not `songs[i]`.
- Returning the internal list directly (an alias) is the classic "encapsulation broken" deduction.

7. Variation C: Class With an Array Field

```
public class ScoreBoard {
    private int[] scores;
    private int filled;      // how many slots are actually used

    public ScoreBoard(int capacity) {
        scores = new int[capacity];
        filled = 0;
    }

    public boolean addScore(int s) {
        if (filled >= scores.length) return false;
        scores[filled] = s;
        filled++;
        return true;
    }

    public double average() {
        if (filled == 0) return 0.0;
        int sum = 0;
        for (int i = 0; i < filled; i++) sum += scores[i];
        return (double) sum / filled;     // cast BEFORE dividing
    }
}
```

- Arrays are fixed-size: carry a `filled` counter whenever the array may be partially used.
- `scores.length` (field, no parens) versus `songs.size()` (method, with parens) is the #1 point killer.
- `(double) sum / filled` - otherwise you get integer division.

8. Variation D: Static Utility Class

```
public class Temperature {
    private Temperature() { }        // no objects needed

    public static double toCelsius(double f) {
        return (f - 32) * 5.0 / 9.0;
    }

    public static boolean isFreezing(double f) {
        return toCelsius(f) <= 0;
    }
}
```

Static methods may only touch `static` fields and other `static` methods - never `this`.

9. Top Traps When Writing a Class

Trap	Symptom	Fix
`void` on a constructor	`CE: invalid method declaration; return type required`	Delete the return type
Constructor name != class name	Treated as a method; `new` fails	Rename to match class exactly
Forgot to initialize a field	`NullPointerException` at runtime	Assign every field in the constructor
`this` in a static method	`CE: non-static variable this cannot be referenced`	Make the method non-static, or drop `this`
`this.x = x` reversed to `x = this.x`	Field silently stays `0`/`null`	Assignment target is always `this.field`
Missing `return` on one path	`CE: missing return statement`	Every branch of a non-void method must return
`==` on Strings	Compares addresses, false negatives	`.equals()`
Returning internal collection	Caller mutates your data	Return a copy
`length()` on an array	`CE: cannot find symbol`	`arr.length` - no parens

10. Self-Check Before You Move On

- [ ] Every field is `private`, typed, and assigned in the constructor.
- [ ] Constructor has no return type and matches the class name.
- [ ] Each method signature matches what the prompt asks for, character for character.
- [ ] Non-void methods return on **every** path; `void` methods return nothing.
- [ ] `toString()` returns a `String` and prints nothing.
- [ ] Static context never touches `this` or instance fields.
- [ ] Collections are constructed when declared or in the constructor, and returned as copies.
- [ ] Ran one sample object by hand: field values correct after construction and after each method.